

Hub

Uživatelský manuál

Backend systému pro spouštění webových aplikací (her)

22. května 2026

Obsah

1	Přehled systému	3
1.1	Tok požadavku	3
1.2	REST API – přehled endpointů	3
2	Nasazení systému Hub na server	4
2.1	Požadavky	4
2.2	Stahování repozitářů	4
2.3	Konfigurace prostředí	4
2.4	Vytvoření sdílené Docker sítě	5
2.5	Spuštění systému	5
2.6	Instalační skript	5
3	Integrace aplikace (hry) do systému Hub	7
3.1	Požadavek 1: Práce za reverse proxy	7
3.2	Požadavek 2: Docker kontejner a sdílená síť	7
3.3	Požadavek 3: Registrace u backendu	8
3.3.1	Registrační skript	8
3.3.2	Struktura JSON záznamu hry	9
3.3.3	Integrace skriptu do Dockerfile	10
3.4	Přehled proměnné prostředí pro herní kontejner	10
4	Příklad – kompletní integrace hry	11
5	Odregistrování hry	12
6	Řešení problémů	13

1 Přehled systému

Hub je centrální backendová služba, která zajišťuje přístup klientů k webovým aplikacím (hrám) prostřednictvím **reverse proxy**. Celý systém se skládá ze tří vrstev:

1. **Backend (hub-backend)** – Go aplikace; obsluhuje REST API pro registraci her a provádí reverse proxy na příslušnou herní službu.
2. **Frontend (hub-frontend)** – Webové rozhraní pro uživatele; zobrazuje seznam dostupných her.
3. **Herní kontejnery** – Libovolné aplikace/hry běžící v Docker kontejnerech, připojené ke sdílené síti **hub-shared**.

1.1 Tok požadavku

1. Prohlížeč odešle požadavek na adresu `https://DOMÉNA/games/id/...`
2. Backend najde registrovanou hru podle `id`.
3. Požadavek je přeposlán na interní adresu herního kontejneru (např. `http://mygame:8080`).
4. Odpověď hry je vrácena prohlížeči.

Požadavky na ostatní cesty (vše mimo `/games/` a `/api/`) jsou přeposílány na frontend službu.

1.2 REST API – přehled endpointů

Metoda	Cesta	Popis
GET	<code>/api/games</code>	Vrátí seznam registrovaných her
POST	<code>/api/games</code>	Zaregistruje novou hru
DELETE	<code>/api/games/{id}</code>	Odregistruje hru
ANY	<code>/games/{id}/...</code>	Reverse proxy na danou hru
ANY	<code>/</code>	Reverse proxy na frontend

2 Nasazení systému Hub na server

2.1 Požadavky

- Docker Engine ≥ 24 a Docker Compose Plugin (`docker compose`)
- Git
- Doménové jméno směřující na server (pro produkci s HTTPS)
- Otevřené porty 80 a 443

2.2 Stažení repozitářů

Repozitáře backendu a frontendu **musí ležet vedle sebe** ve stejném nadřazeném adresáři, protože `docker-compose.yml` backendu odkazuje na frontend cestu `../hub-frontend`.

Ukázka 1: Klonování repozitářů

```
1 mkdir hub && cd hub
2 git clone https://github.com/<org>/hub-backend.git backend
3 git clone https://github.com/<org>/hub-frontend.git
  hub-frontend
```

Výsledná struktura adresářů:

```
hub/
| - backend/           # repozitář backendu
+ - hub-frontend/      # repozitář frontendu
```

2.3 Konfigurace prostředí

Zkopírujte vzorový soubor a upravte hodnoty:

```
1 cd backend
2 cp .env.example .env
3 nano .env           # nebo vim, code, ...
```

Klíčové proměnné:

Proměnná	Výchozí hodnota	Popis
APP_ENV	development	Režim běhu (production / development)
DOMAIN	–	FQDN domény (např. hub.example.com)
CERT_EMAIL	–	E-mail pro Let's Encrypt
CERT_CACHE_DIR	/certs	Adresář pro cache TLS certifikátů
FRONTEND_TARGET	http://frontend:5173	Interní URL frontend kontejneru
HTTP_ADDR	:80	HTTP port (redirect + ACME challenge)
HTTPS_ADDR	:443	HTTPS port

Příklad `.env` pro produkci:

Ukázka 2: Příklad `.env` pro produkci

```
1 APP_ENV=production
2 DOMAIN=hub.example.com
3 CERT_EMAIL=admin@example.com
4 CERT_CACHE_DIR=/certs
5 FRONTEND_TARGET=http://frontend:5173
6 HTTP_ADDR=:80
7 HTTPS_ADDR=:443
```

2.4 Vytvoření sdílené Docker sítě

Sdílená síť `hub-shared` musí existovat **před** spuštěním `compose`, protože je deklarována jako `external: true`.

```
1 docker network create hub-shared
```

Tuto síť stačí vytvořit jednou; po restartu serveru ji Docker zachová.

2.5 Spuštění systému

Ukázka 3: Sestavení a spuštění hub (backend + frontend)

```
1 cd backend
2 docker compose up --build -d
```

Příznak `-build` zajistí sestavení obrazů ze zdrojového kódu. Příznak `-d` spustí služby na pozadí.

Poznámka

Při prvním spuštění v produkčním režimu backend automaticky získá TLS certifikát od Let's Encrypt. Port 80 musí být dostupný z internetu kvůli ACME HTTP-01 challenge.

2.6 Instalační skript

Pro zjednodušení nasazení je k dispozici skript `install-hub.sh` (viz příloha nebo kořen repozitáře). Skript:

1. Naklonuje oba repozitáře do správné struktury.
2. Vytvoří Docker síť `hub-shared` (pokud neexistuje).
3. Zkopíruje `.env.example` → `.env` (pokud `.env` neexistuje).
4. Spustí `docker compose up -build -d`.

Použití:

```
1 bash install-hub.sh
2 # nebo
3 chmod +x install-hub.sh && ./install-hub.sh
```

Upozornění

Před spuštěním skriptu upravte `.env` s hodnotami pro vaše prostředí (zejména `DOMAIN` a `CERT_EMAIL`).

3 Integrace aplikace (hry) do systému Hub

Tato kapitola popisuje, jak vytvořit novou hru/aplikaci kompatibilní s Hub systémem. Pro úspěšnou integraci musí aplikace splňovat tři podmínky:

1. Běžet v Docker kontejneru.
2. Být připojena na sdílenou síť `hub-shared`.
3. Registrovat se u backendu přes REST API při startu.

3.1 Požadavek 1: Práce za reverse proxy

Backend přeposílá požadavky na hru a **ořezává** prefix `/games/{id}` z URL. Aplikace tedy vidí cestu **od kořene** (`/`).

Poznámka

Příklad: uživatel navštíví `https://hub.example.com/games/chess/board` – hra obdrží požadavek na cestu `/board`.

Důsledky pro aplikaci:

- Všechny interní odkazy (HTML `href`, JS `fetch`, CSS `url()`) musí být **relativní** nebo začínat `/games/{id}/`.
- Statické soubory (JS, CSS, obrázky) musí být servovány ze stejné cesty jako HTML – ideálně z kořene.
- Pokud aplikace používá klientský router (SPA), musí server vracet `index.html` pro všechny neznámé cesty (catch-all).

Příklad konfigurace Nginx pro SPA aplikaci:

Ukázka 4: `nginx.conf` – catch-all pro SPA

```

1 server {
2     listen 80;
3     root /usr/share/nginx/html;
4     index index.html;
5
6     location / {
7         try_files $uri $uri/ /index.html;
8     }
9 }
```

3.2 Požadavek 2: Docker kontejner a sdílená síť

Hra musí být kontejnerizována a připojena do sítě `hub-shared`. Hub i hra pak mohou komunikovat přes Docker DNS (jménem kontejneru / služby).

Minimální `docker-compose.yml` pro hru:

Ukázka 5: docker-compose.yml herní aplikace

```

1 services:
2   mygame:
3     build: .
4     expose:
5       - "80"           # port exponovan jen uvnitř site, NE na
                        # hostiteli
6     networks:
7       - hub-shared
8     restart: unless-stopped
9
10  networks:
11    hub-shared:
12      external: true    # připojení na existující sdílenou síť hubu

```

Upozornění

Používejte **expose** (interní port), nikoli **ports** (publikovaný port na hostiteli). Hra nemá být dostupná přímo z internetu – veškerý přístup musí procházet přes Hub.

3.3 Požadavek 3: Registrace u backendu

Při startu kontejneru se hra musí zaregistrovat u Hub backendu pomocí HTTP POST na `/api/games`. Tuto registraci provádí startovací skript.

3.3.1 Registrační skript

Vytvořte soubor `register-and-run.sh`:

Ukázka 6: register-and-run.sh

```

1 #!/bin/sh
2 set -eu
3
4 BACKEND_URL=${GAME_HUB_URL:-http://backend:8080}
5 ENDPOINT="$BACKEND_URL/api/games"
6
7 # Upravte pole dle své aplikace
8 payload='{
9   "id":           "mygame",
10  "name":          "My Game",
11  "target":        "http://mygame:80",
12  "status":        "online",
13  "icon":          "nejaka emoji",
14  "author":        "Jmeno autora",
15  "description":   "Strucny popis hry.",
16  "image":         "https://example.com/mygame-icon.png"
17 }',
18

```



```

19 max_retries=30
20 sleep_seconds=2
21 attempt=1
22
23 while [ $attempt -le $max_retries ]; do
24     echo "[register] Pokus $attempt: POST $ENDPOINT"
25     http_code=$(curl -s -o /tmp/resp -w "%{http_code}" \
26         -X POST "$ENDPOINT" \
27         -H "Content-Type: application/json" \
28         -d "$payload" || true)
29
30     if [ "$http_code" = "201" ] || [ "$http_code" = "200" ]; then
31         echo "[register] Registrace uspela (HTTP $http_code)"
32         break
33     fi
34
35     echo "[register] Selhani (HTTP $http_code), dalsi pokus za
36         ${sleep_seconds}s"
37     attempt=$((attempt + 1))
38     sleep $sleep_seconds
39 done
40 # Spustit vlastni aplikaci (nginx, node, ...)
41 exec nginx -g "daemon off;"

```

Poznámka

Pole `target` musí obsahovat interní Docker adresu hry – tedy název služby a port, pod kterým ji Hub backend dosáhne uvnitř sítě `hub-shared`. Například `http://mygame:80`.

3.3.2 Struktura JSON záznamu hry

Pole	Typ	Popis
<code>id</code>	string	Unikátní identifikátor (URL-safe, bez mezer)
<code>name</code>	string	Zobrazovaný název hry
<code>target</code>	string	Interní URL kontejneru hry (Docker DNS)
<code>status</code>	string	Stav hry (<code>online</code> / <code>offline</code>)
<code>icon</code>	string	Emoji nebo krátký symbol
<code>author</code>	string	Jméno autora
<code>description</code>	string	Krátký popis
<code>image</code>	string	URL náhledového obrázku (volitelné)

3.3.3 Integrace skriptu do Dockerfile

Ukázka 7: Dockerfile herní aplikace (zkrácený příklad)

```

1 FROM nginx:alpine
2
3 # curl je potřeba pro registracni skript
4 RUN apk add --no-cache curl
5
6 # Zkopirujte vysledny web do Nginx rootu
7 COPY dist/index.html /usr/share/nginx/html/index.html
8 COPY nginx.conf /etc/nginx/conf.d/default.conf
9
10 # Registracni skript
11 COPY register-and-run.sh /register-and-run.sh
12 RUN chmod +x /register-and-run.sh
13
14 EXPOSE 80
15
16 # Skript nejprve zaregistruje hru, pak spusti nginx
17 CMD ["/register-and-run.sh"]

```

3.4 Přehled proměnné prostředí pro herní kontejner

Proměnná	Výchozí hodnota	Popis
GAME_HUB_URL	http://backend:8080	URL Hub backendu uvnitř Docker sítě

4 Příklad – kompletní integrace hry

Tato sekce shrnuje celý postup na ukázkovém příkladu hry *Chess*.

Krok 1. Ujistěte se, že síť `hub-shared` existuje:

```
docker network create hub-shared
```

Krok 2. Vytvořte registrační skript `register-and-run.sh` (viz [sekci 3.3.1](#)) s hodnotami pro vaši hru (`id=chess`, `target=http://chess:80`, ...).

Krok 3. Nakonfigurujte Nginx (`nginx.conf`) s catch-all pro SPA (viz [sekci 3.1](#)).

Krok 4. Napište `Dockerfile` zahrnující `curl`, registrační skript a Nginx (viz [sekci 3.3.3](#)).

Krok 5. Vytvořte `docker-compose.yml` hry připojené na `hub-shared`:

```
services:
  chess:
    build: .
    expose:
      - "80"
    networks:
      - hub-shared
    restart: unless-stopped

networks:
  hub-shared:
    external: true
```

Krok 6. Spusťte hru:

```
docker compose up --build -d
```

Krok 7. Ověřte registraci:

```
curl https://hub.example.com/api/games
```

V odpovědi by měla být hra `chess` se stavem `online`.

Krok 8. Hra je dostupná na adrese `https://hub.example.com/games/chess`.

5 Odregistrování hry

Pokud chcete hru z Hubu odebrat (např. při odstavení kontejneru), zavolejte DELETE endpoint:

```
1 curl -X DELETE https://hub.example.com/api/games/chess
```

Odpověď: 204 No Content. Hra zmizí ze seznamu dostupných her.

Poznámka

Registrace her je ukládána do souboru `data/games.json` uvnitř backend kontejneru. Po restartu backendu se dříve registrované hry automaticky obnoví. Herní kontejnery se musí po restartu samy znovu zaregistrovat (registrační skript toto zajišťuje automaticky při spuštění kontejneru).

6 Řešení problémů

Problém	Řešení
Hra se neregistruje	Zkontrolujte, zda je kontejner hry ve stejné síti jako backend (<code>hub-shared</code>). Ověřte DNS: <code>docker exec <kontejner> ping backend</code> .
404 na <code>/games/id</code>	Hra není registrována. Zkontrolujte logy kontejneru hry a výstup registračního skriptu.
Aktiva (CSS/JS) se nenačítají	Statické soubory musí být na relativních cestách. Ujistěte se, že Nginx vrací <code>index.html</code> pro neznámé cesty (<code>catch-all</code>).
Let's Encrypt selhává	Port 80 musí být dostupný z internetu. Zkontrolujte firewall a zda <code>DOMAIN</code> správně ukazuje na server.
Síť <code>hub-shared</code> neexistuje	Spusťte <code>docker network create hub-shared</code> před <code>docker compose up</code> .